```
beforeAll(() => provider.setup());

describe('When a query to list all movies on /graphql is made', () => {
  beforeAll(() => {

    const graphqlQuery = new GraphQLInteraction()
      .uponReceiving('a movies request')
      .withQuery(
        `
        query MoviesQuery {
          movies {
            id
            name
            year
          }
        }
        `
      )
      .withOperation('MoviesQuery')
      .withVariables({})
      .withRequest({
        method: 'POST',
        path: '/graphql',
      })
      .willRespondWith({
        status: 200,
        headers: {
          'Content-Type': 'application/json; charset=utf-8',
        },
        body: {
          data: {
            movies: eachLike(EXPECTED_BODY),
          },
        },
      });
    return provider.addInteraction(graphqlQuery);

  })
});
```

With this interaction, we're creating a new instance of `GraphQLInteraction`, calling the `uponReceiving` method and passing in the scenario name as a string. In this example, we say the scenario name is `'a movies request'`. Next, we call the `withQuery` method and pass in the GraphQL query to retrieve all movies. Because we have provided `MoviesQuery` as our operation name, which is optional, we also call the method `withOperation` and pass the operation name. The `withVariables` method represents any variables we need to pass to our query, which in this case is none, so we pass an empty object. We call the `withRequest` method to make the request, passing in the `/graphql` endpoint.

To represent the query response details from the mock provider, we call the `willRespondWith` API and pass in the response details. In this example, the